

NOIP 模拟赛

szt100309&Gold14526

时间： 08:00 ~ 11:50

题目名称	排序	双人	Rplexq	染色
题目类型	传统型	传统型	传统型	传统型
目录	sort	couple	rplexq	paint
可执行文件名	sort	couple	rplexq	paint
输入文件名	sort.in	couple.in	rplexq.in	paint.in
输出文件名	sort.out	couple.out	rplexq.out	paint.out
每个测试点时限	1.0 秒	4.0 ~ 7.0 秒	1.0 ~ 3.0 秒	2.0 秒
内存限制	64 MiB	128 MiB	128 ~ 2048 MiB	1024 MiB
测试包数量	6	6	13	10
测试包是否等分	否	否	否	否

提交源程序文件名

对于 C++ 语言	sort.cpp	couple.cpp	rplexq.cpp	paint.cpp
-----------	----------	------------	------------	-----------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 `main()` 返回值类型必须为 `int`，程序正常结束时的返回值必须是 0。
3. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
4. 选手提交的程序源文件必须不大于 100KB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 只提供 Linux 格式附加样例。
7. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。

排序 (sort)

原题: [P14232 \[COI 2011\] 排序 / SORT](#)

以下认为自环不算置换环。

若不存在置换环, 显然无需操作。

否则每次操作可以消除一个置换环 (对于环 $k_{1 \sim l}$, 进行一次操作 $k_{1 \sim l}$ 即可), 最优解上界为置换环数量。

当置换环数量大于 1 时, 先对所有点进行一次操作 (设有 c 个置换环, 第 i 个环上点为 $k_{i,1 \sim l_i}$, 则操作 $k_{1,1 \sim l_1} + k_{2,1 \sim l_2} + \cdots + k_{c,1 \sim l_c}$, 其中 $+$ 表示拼接), 可证操作后剩下一个环, 再操作一次即可。

显然这种情况下一次操作无法完成操作, 因此取到下界。

容易做到 $O(n)$ 。

[代码](#)

[参考](#)

双人 (couple)

原题: [P14261 兄妹 \(Ver. 2\)](#)

显然最优解中一行只会属于一个集合, 因此令 a_x 表示第 x 行 y 坐标的最大值。

对于划分出的一个行号集合为 S , 显然其最小代价为 $2 \max S + 2 \sum_{u \in S} a_u - \max_{u \in S} (u + a_u)$ 。

令 $p = \operatorname{argmax}_{u \in S} a_u$, $q = \max S$, 显然 $p \leq q$, 则具体构造为 $(0, 0) \rightarrow \{(i, 0) \rightarrow (i, a_i) \rightarrow (i, 0)\}_{i=1}^{p-1} \rightarrow (p, 0) \rightarrow \{(i, 0) \rightarrow (i, a_i) \rightarrow (i, 0)\}_{i=p+1}^q \rightarrow (p, 0) \rightarrow (p, a_p)$ 。

分为三段: $[0, p], (p, q], (q, +\infty)$, 令 $f_{i,j,0/1/2,0/1/2}$ 表示 $a_1 \sim i$, 第一个人走了 j 的时间, 第一个人第 $0/1/2$ 段, 第二个人第 $0/1/2$ 段, 第二个人走的最小时间。

转移是容易的, 时间复杂度 $O(\sum (\max r_i)^2 \cdot \max c_i)$, 需要注意常数。

[代码](#)

[参考](#)

Rplexq (rplexq)

原题: P6782 [Ynoi2008] rplexq

以下 $\deg(u)$ 表示结点 u 的儿子数量。

令 $C(x, l, r)$ 表示子树 x 中编号在 $[l, r]$ 内的数量, 询问 (l, r, u) 相当于求

$$\frac{C(u, l, r)^2 - \sum_{v \in \text{son}(u)} C(v, l, r)^2}{2}$$

设阈值 S 。

对于询问 (l, r, x) , 若 $\deg(x) \leq S$, 则转化为 $O(Sq)$ 个 $C(x, l, r)$ 的查询, 离线后转化为二维数点, 并用 $O(\sqrt[n]{n})$ 修改、 $O(c)$ 查询的分块即可做到 $O(cqS + n\sqrt[n]{n})$ 。

若 $\deg(x) > S$, 将儿子按子树大小从大到小排序, 称一个结点前 S 大儿子为重儿子, 其余为轻儿子, $C(x, l, r)$ 和 $C(w, l, r)$ (其中 w 取遍 x 的重儿子) 处理方式同上, 时间复杂度相同, 轻儿子部分挂在结点 x 上。

依次考虑每个存在轻儿子的结点 x (显然 x 上存在询问的必要条件为 $\deg(x) > S$), 枚举其每个轻儿子 y , 将在子树 y 内且编号在 $[l, r]$ 内的点染上颜色 y , 并将这些点按编号从小到大排序, 则转化为查询区间中所有等价类大小的平方和, 使用莫队即可, 设结点 x 的轻子树大小之和为 L_x , 结点 x 上询问数量为 Q_x , 则时间复杂度 $O(\sum_{i=1}^n L_i \sqrt{Q_i})$ 。

引理. 所有轻子树大小之和为 $O(n \log_S n)$ 的。

证明:

1. 若一个点为其父亲的轻儿子, 称其为关键点。
2. 对于任意点 u , 找到它到根的链上第一个关键点 u_1 , 显然子树 u_1 的大小至少为 $S+2$ 。
3. 找到 u_1 的父亲到根的链上的第一个关键点 u_2 , 显然子树 u_2 大小至少为 $(S+2)(S+1)+1$ 。
4. 以此类推, 子树 u_p 的大小至少为 $\sum_{i=0}^p (S+1)^i = \frac{(S+1)^{p+1}-1}{S}$ 。
5. 由于子树大小不超过 n , 因此 p 至多为 $O(\log_S n)$ 。
6. 从而所有轻子树大小之和为 $O(n \log_S n)$ 的。

一个取到上界的构造: 建立完全 $2S$ 叉树, 则每层有一半的结点为关键点, 大小总和为 $\log_{2S} n \times \frac{n}{2} = O(n \log_S n)$ 。

定理. $O(\sum_{i=1}^n L_i \sqrt{Q_i}) \leq O(n \sqrt{q \log_S n})$ 。

证明:

1. 根据柯西不等式，有：

$$\sum_{i=1}^n L_i \sqrt{Q_i} \leq \sqrt{\sum_{i=1}^n L_i^2 \times \sum_{i=1}^n Q_i} \leq \sqrt{q \sum_{i=1}^n L_i^2}$$

2. 而：

$$\sum_{i=1}^n L_i^2 \leq n \sum_{i=1}^n L_i = n \times O(n \log_S n)$$

3. 从而 $O(\sum_{i=1}^n L_i \sqrt{Q_i}) \leq O(n \sqrt{q \log_S n})$ 。

从而这部分时间复杂度为 $O(n \sqrt{q \log_S n})$ (实际上似乎仍然卡不满, 因为此时要求 $u_{1 \sim \log_S n} = n$)。

以上算法总时间复杂度 $O(cqS + n\sqrt[n]{n} + n\sqrt{q \log_S n})$, 空间复杂度 $O(n + qS)$ 无法接受。

考虑第一部分扫描线时, 初始将每个待查询部分的第一个儿子挂在相应位置, 每次查询完某个子树的一个儿子后将下一个儿子挂在相应位置, 并释放当前的空间, 这样时间复杂度不变但空间复杂度线性。

总时间复杂度 $O(cqS + n\sqrt[n]{n} + n\sqrt{q \log_S n})$, 空间复杂度 $O(n + q)$, 取 $S = O(\sqrt[n]{n}) \wedge S \leq \sqrt{n}, c = O(1) \wedge c \geq 2$ 即可做到 $O(n\sqrt{q})$ 。

由于 $O(cqS)$ 部分常数较大, 建议 S 取较小的值。std 中 $c = 2, S = \sqrt[10]{n}$ 。

[代码](#)

[参考](#)

染色 (paint)

原题: [P7295 \[USACO21JAN\] Paint by Letters P](#)

每个格子建立一个点, 若相邻两个格子颜色相同则连边, 显然得到一张平面图。

根据平面图欧拉公式 $B = V - E + R$ (B 为连通块数, V 为点数, E 为边数, R 为有界面数), 转化为求出给定子矩形的导出子图的点数、边数、有界面数。

点数显然为长乘以宽, 边数容易二维前缀和预处理。关键在于计算无界面数。

建立其对偶图 (原图每个 2×2 的子矩阵中心建立一个点, 若对偶图中相邻两点之间的边与原图的边无交则相连), 则原图的一个面就是对偶图的一个连通块, 转化为查询完全位于子矩阵中的连通块数量。

一种方法为求出每个连通块的外接矩形, 从而每次询问转化为求完全包含于给定矩形中的矩形数量, 显然为四维偏序, 时间复杂度 $O((nm + q) \log^3(nm + q))$, 得分未知。

更巧妙的算法为: 对于每个连通块选择一个关键点, 每次询问时先统计出查询范围内的关键点数量, 然后绕着矩形的边缘走, 枚举紧靠着边缘外侧的一圈点 (共 $O(n + m)$ 个), 若它们所在的连通块的关键点在查询范围内则答案减一, 注意不要重复计算。容易做到 $O(nm + q(n + m))$ 。

[代码](#)

[参考](#)

总结

预计 $100 + (\geq 65) + (\geq 46) + (\geq 39)$ 。

订正要求 $100 + 100 + (\geq 97) + 100$ 。